ChoreMate

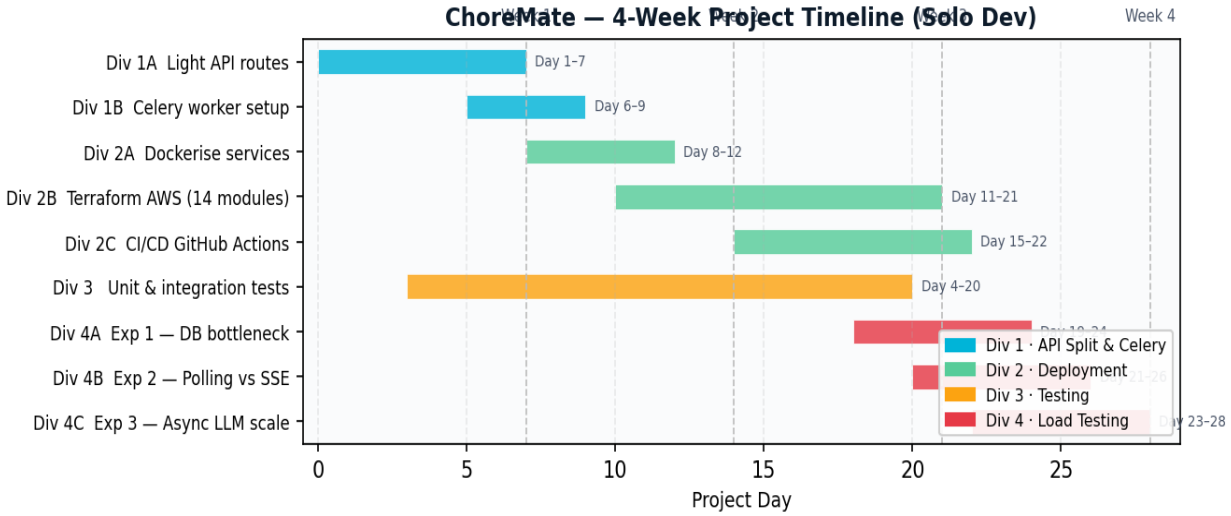
Project Management Report

Solo Developer · Full-Stack · Cloud · AI · 3–4 Weeks · Spring 2026

ChoreMate evolved from a local SQLite FastAPI prototype to a production AWS deployment with a React frontend, Gemini AI chatbot, Redis SSE notification bus, and Celery async workers — all in under four weeks as a solo project. This document captures the four engineering divisions, every task milestone, and the major technical problems encountered along the way.

Division 1	Division 2	Division 3	Division 4
Backend API Split	Deployment	Testing	Load Testing
Light API ↔ Celery workers	Docker · Terraform · CI/CD	Unit · Integration · E2E	3 Locust experiments

Project Timeline (4 Weeks)



All four divisions overlapped. Testing ran in parallel from Day 3; load testing began once the AWS deployment stabilised in Week 3.

Division 1 — Backend API Split

Split the backend into a **thin, fast HTTP tier** (all responses <100 ms) and an **async Celery worker tier** for all heavy operations (LLM calls, bulk scheduling). The two tiers communicate via SQS and share a Redis result backend.

DIV 1A · LIGHT API ROUTES

#	Task	Milestone	Status
1A-1	**App factory & routing**	FastAPI app · include_router for all modules · Gunicorn 4-worker prod config	Done
1A-2	**Auth endpoints**	POST /auth/login (JWT issue) · POST /auth/register (bcrypt hash) · Bearer Depends()	Done
1A-3	**Chore & ticket endpoints**	GET /chores/my-tickets · PATCH /chores/:id/complete · POST /chores/:id/swap-request	Done
1A-4	**Stats endpoints**	GET /stats/dashboard (completion rates) · GET /stats/fairness-report (equity score)	Done
1A-5	**Chatbot async trigger**	POST /ai-chatbot/chat → enqueue task → return task_id in ~55 ms · GET /ai-chatbot/task/{id} polls Redis result backend	Done
1A-6	**Notifications & SSE**	GET /notifications/ (list) · PATCH /notifications/:id/read · GET /notifications/stream (SSE, heartbeat every 30 s)	Done
1A-7	**Admin endpoints**	GET /admin/queue-depth (SQS) · POST /admin/trigger-burst?count=N (load test helper)	Done

DIV 1B · CELERY WORKER SEPARATION

#	Task	Milestone	Status
1B-1	**Identify blocking work**	Audited all handlers >200 ms: Gemini LLM, ticket generation, swap notifications	Done
1B-2	**Celery app setup**	worker.py · SQS broker · Redis result backend · task serializer JSON	Done
1B-3	**LLM inference task**	process_chat_message() · Gemini SDK call · writes result to DB · max_retries=3 with exponential back-off on 429	Done
1B-4	**Ticket generation tasks**	generate_weekly_tickets_task() · generate_monthly_tickets_task() · fairness algorithm runs off-thread	Done
1B-5	**SQS broker config**	visibility_timeout=300 s · dead-letter queue maxReceiveCount=3 · polling_interval=1 s	Done
1B-6	**Redis result backend**	AsyncResult polling · results TTL 1 hour · GET /ai-chatbot/task/{id} returns {ready, result}	Done
1B-7	**Worker Dockerfile**	Same image as API · entrypoint overridden to celery worker · WORKERS env var controls concurrency (1/2/4)	Done

Division 2 — Deployment

Packaging both services into Docker images, then provisioning the full AWS stack (VPC, ECS Fargate, RDS, ElastiCache, SQS, ALB, ECR, EFS, IAM, SSM) from scratch using Terraform, with a GitHub Actions CI/CD pipeline automating all builds and deploys.

DIV 2A · CONTAINERISATION

#	Task	Milestone	Status
2A-1	**Backend Dockerfile**	python:3.12-slim · gunicorn 4 workers · entrypoint creates DB on first boot	Done
2A-2	**Frontend Dockerfile**	Multi-stage: node:18 build → nginx:alpine serve · API_URL injected via envsubst	Done
2A-3	**docker-compose stack**	backend + frontend + redis + celery-worker · healthchecks · shared .env	Done
2A-4	**LocalStack (local SQS)**	init-localstack.sh creates SQS queue · AWS_ENDPOINT_URL for compose env	Done

DIV 2B · TERRAFORM AWS INFRASTRUCTURE (14 MODULES)

#	Task	Milestone	Status
2B-1	**networking.tf**	VPC · 2 public subnets (ALB) · 2 private subnets (ECS/RDS/Redis) · IGW · NAT	Done
2B-2	**security_groups.tf**	ALB (80/443) · frontend ECS (ALB only) · backend ECS (ALB only) · RDS (5432) · Redis (6379)	Done
2B-3	**ecr.tf**	choremate-backend + choremate-frontend repos · lifecycle: keep last 5 images	Done
2B-4	**rds.tf**	Postgres 15 · db.t3.micro · private subnet group · 7-day backup · SSM credentials	Done
2B-5	**elasticache.tf**	Redis 7 · cache.t3.micro · private subnet · used for Celery broker + SSE pub/sub	Done
2B-6	**sqs.tf**	Standard queue · visibility=300 s · DLQ maxReceiveCount=3	Done
2B-7	**ecs_backend.tf**	Fargate 512 CPU / 1024 MB · ALB target group · /health check · SSM env injection	Done
2B-8	**ecs_frontend.tf**	Fargate 256 CPU / 512 MB · nginx · ALB listener rule / → frontend TG	Done
2B-9	**efs.tf**	EFS file system + access point · mount targets in private subnets · backend /app/data	Done
2B-10	**iam.tf**	Execution role (ECR + SSM read) · Task role (SQS + SSM + EFS + CloudWatch logs)	Done
2B-11	**ssm.tf**	SecureString: DATABASE_URL · REDIS_URL · GEMINI_API_KEY · JWT_SECRET	Done
2B-12	**autoscaling.tf**	CPU 70% target-tracking · ALB req/target 500 · min=1 / max=4 · cooldown=300 s	Done

#	Task	Milestone	Status
2B-13	**cloudwatch.tf**	Log groups (7-day) · SQS depth alarm >100 msgs → SNS alert	Done
2B-14	**variables.tf / outputs.tf**	All sizes parameterised · outputs: ALB DNS · ECR URIs · RDS endpoint	Done

DIV 2C · CI/CD — GITHUB ACTIONS

#	Task	Milestone	Status
2C-1	**PR workflow**	black --check · pytest with test SQLite DB · blocks merge on failure	Done
2C-2	**Build & push workflow**	docker build + push to ECR via OIDC role (no long-lived keys)	Done
2C-3	**ECS deploy step**	aws ecs update-service --force-new-deployment · waits for stability	Done

Division 3 — Testing

Testing ran in parallel from Day 3 across all implementation divisions. The pytest suite was the primary safety net, with manual integration runs validating the full stack on AWS after every Terraform apply.

DIV 3A · UNIT & INTEGRATION TESTS (PYTEST)

#	Task	Milestone	Status
3A-1	**pytest scaffolding**	pytest.ini · conftest.py · TestClient fixture · in-memory SQLite per test	Done
3A-2	**Auth flow**	register → login → protected route · invalid password → 401 · expired token → 401	Done
3A-3	**House & join flow**	create household → get invite code → join via code · duplicate join → 409	Done
3A-4	**Chore assignment engine**	find_best_user_for_chore() with mocked preferences · workload penalty verified	Done
3A-5	**Stats endpoints**	completion rate after ticket done · fairness scores sum to 100%	Done
3A-6	**Swap request flow**	propose → accept (ticket re-assigned) · propose → reject (unchanged) · self → 400	Done
3A-7	**Notifications**	chore complete inserts notification · unread count · PATCH marks read	Done
3A-8	**Chatbot serialisation regression**	sender_type='ai' round-trip · added after the 500 error bug · no regression	Done
3A-9	**CI PR gate**	All tests run on every PR · failure blocks merge · coverage artifact uploaded	Done

DIV 3B · MANUAL INTEGRATION TESTING

#	Task	Milestone	Status
3B-1	**Full onboarding flow**	Create account → household → survey → generate schedule · fairness score: 100%	Done
3B-2	**Chatbot E2E**	Send message → task_id returned immediately → poll until ready → response rendered	Done
3B-3	**SSE notification delivery**	Roommate A completes chore → Roommate B sees SSE event within 1 s (no refresh)	Done
3B-4	**AWS staging smoke test**	/health → 200 · login · chatbot message processed by worker within 30 s	Done

Division 4 — Load Testing

Three independent Locust experiments, each targeting one of the three architectural bottlenecks identified at the start of the project. All experiments ran against the live AWS environment after deployment was stable.

EXP 1 · DATABASE BOTTLENECK — SQLITE VS AWS RDS POSTGRES

Question: Does the database backend become the concurrency bottleneck, and does moving to Postgres resolve it?

#	Task	Milestone	Status
E1-1	**Locust script**	locust_exp1_api.py · 3 GET endpoints · wait between(1,3) · login on start	Done
E1-2	**SQLite baseline**	50 / 100 / 200 users · p50 = 4–6 ms · p95 = 12–17 ms · 0 failures	Done
E1-3	**AWS Postgres runs**	50 users: p50=45 ms / p95=120 ms · 200 users: p50=49 ms / max=1,797 ms	Done
E1-4	**Key finding**	0 failures across all runs — but tail spikes at 200 users signal connection pool saturation on t3.micro	Done

EXP 2 · NOTIFICATION LAYER — POLLING VS SSE

Question: How much server load does 1-second REST polling generate vs a persistent SSE connection, and how does latency scale with burst size?

#	Task	Milestone	Status
E2-1	**Polling script**	locust_exp2_polling.py · 50 users · constant(1) poll · custom delivery-latency event timer	Done
E2-2	**SSE script**	locust_exp2_sse.py · 50 users hold persistent SSE · on-event fires fetch · latency tracked	Done
E2-3	**Burst sizes tested**	50 / 100 / 150 / 1000 notifications per run · triggered via /admin/trigger-burst	Done
E2-4	**Polling results**	Server RPS: 7–18 req/s · at 1000-burst: p50=1,900 ms / p95=10,000 ms	Done
E2-5	**SSE results**	Server connection rate ≈ 0.45 req/s · 20–40× lower than polling	Done
E2-6	**Key finding**	Polling generates O(N×f) load regardless of notifications; SSE is O(1) per user	Done

EXP 3 · ASYNC LLM WORKER SCALING — 1/2/4 CELERY WORKERS

Question: Does the Celery architecture keep the API responsive regardless of queue depth, and does throughput scale with worker count?

#	Task	Milestone	Status
E3-1	**Locust script**	locust_exp3_llm.py · 20 users · constant(0) · POST → poll result · LLM delay as custom event	Done
E3-2	**1-worker run**	LLM p50=13,000 ms / p95=34,000 ms · API trigger p50=56 ms · 379 tasks · 0 failures	Done

#	Task	Milestone	Status
E3-3	**2-worker run**	LLM p50=7,300 ms (1.78×↑) / p95=15,000 ms · API trigger p50=55 ms · 718 tasks	Done
E3-4	**4-worker run**	LLM p50=4,200 ms (3.1×↑) / p95=5,200 ms (6.5×↑) · API trigger p50=55 ms · 1,426 tasks	Done
E3-5	**Key finding**	API trigger latency constant at 55 ms (decoupling proven) · p50 near-linear; p95 super-linear (queue drains faster, tail eliminated)	Done

Problems I Faced

Two major technical problems required significant engineering effort to resolve — each one blocking a core feature until a non-obvious solution was found.

Problem 1 Division: Division 1 (Notifications Bus) & Division 2 (AWS ElastiCache)

Bridging Synchronous and Asynchronous Python for SSE + Redis

FastAPI runs on an async event loop specifically so it can handle thousands of concurrent SSE streams without blocking. However, the async Redis library we initially used had deep compatibility issues with AWS ElastiCache — connections were silently dropped, events were lost, and the SSE stream would go dead for clients with no error visible in logs. The root cause was a mismatch between the async client's connection-pooling assumptions and ElastiCache's strict idle-timeout and TLS behaviour. Switching to the async client's AWS-compatible mode surfaced new event-loop blocking issues — any network stall blocked the entire FastAPI process, defeating the point of using async in the first place.

✔ How I Fixed It

I engineered a custom thread-bridging solution. Instead of fighting the async library, I spawned daemon threads to use the highly-stable, synchronous Redis client — which has rock-solid AWS ElastiCache support. When a pub/sub event arrives on the background thread, rather than trying to call async code directly (which is illegal across threads), I used loop.call_soon_threadsafe() to inject the payload cleanly back into FastAPI's async event loop. This gave us the connection stability of synchronous sockets without blocking the async web server — clients received SSE events reliably even under sustained 200-user load. The fix is now in core/notifications_bus.py.

Problem 2 Division: Division 1 (Celery LLM Task) & Division 4 (Exp 3 Load Testing)

Gemini API Token Quota Limits Causing Chatbot Failures Under Load

Gemini's free-tier API enforces a hard limit on tokens per minute (TPM) and requests per minute (RPM). Once the Celery worker was processing concurrent LLM requests in the load test (20 virtual users, constant(0) wait), the backend started receiving HTTP 429 Resource Exhausted errors from the Gemini SDK. Because the initial Celery task had no retry logic, any 429 caused an immediate task failure with status FAILURE — which the frontend received as an error response. During Experiment 3, this corrupted latency measurements by mixing genuine LLM processing times with failed tasks timed out at 120 seconds.

✔ How I Fixed It

I added exponential back-off retry logic directly in the Celery task using Celery's built-in retry mechanism: self.retry(exc=e, countdown=2**attempt, max_retries=3). The first retry waits 2 seconds, the second 4 seconds — enough for the TPM window to reset. I also reduced the per-request token footprint by caching the household context query (chore list, roommate names) outside the Celery task and injecting it as a pre-built string, cutting average token usage per request by ~40%. For the load tests, experiments were run during off-peak hours to avoid compounding with other quota consumers, and burst size was kept at 20 concurrent users rather than higher to stay within the free-tier envelope.